

Formal Specifications

Solving Real-Life Problems with Mathematical Precision

Dr. Abdul Rehman

What is a Formal Specification?

Imagine a blueprint for a skyscraper. It's not just a simple drawing; it's a clear plan showing exactly where every beam and joint should be placed.

Formal Specification is the mathematical "blueprint" for software and systems. It defines *exactly* what the system should do without saying *how* to code it.



| The Cost of Ambiguity



English Language

Vague, interpreted differently by every developer, leading to bugs.



Coding Language

Too detailed. It's easy to get lost in the weeds of implementation.



Formal Math

Precise and concise. It proves the system works before you write a single line of code.

Real-Life Case: The ATM



A simple interaction:

- ✓ Withdraw cash.
- ✓ Update balance.
- ✓ Check limits.

If we get the logic wrong here, people lose money or the bank collapses. English is not enough.

ATM: Formalizing States

Before writing code, we define the "world" of the ATM mathematically:

Variable	Type	Constraint (The Rule)
AccountBalance	Integer	Must be ≥ 0 (No unauthorized debt)
DailyLimit	Integer	Default: \$500
AmountRequested	Integer	Must be > 0 and $\leq$ DailyLimit

| Pre-Conditions: Safety First

A "Pre-condition" is what must be true **before** the operation happens.

Valid Card?

Card must be active and
PIN must be correct.

Enough Money?

AccountBalance $\geq$
AmountRequested

ATM Status?

CashInMachine $\geq$
AmountRequested

Post-Conditions: The Guarantee

A "Post-condition" describes the state of the world **after** the button is pressed.

- ✓ **Balance Update:** $\text{NewBalance} = \text{OldBalance} - \text{AmountRequested}$.
- ✓ **Cash Ejected:** User holds physical cash.
- ✓ **Audit Log:** A record of the transaction exists in the database.



Core Building Blocks

The Language of Specs

| The "What": Sets and Types

Defining Categories

We group everything into sets. For an airline system:

- ✓ **USERS:** {Pilots, Crew, Passengers}
- ✓ **FLIGHTS:** {AA101, BA202, ...}
- ✓ **STATUS:** {OnTime, Delayed, Cancelled}



Math treats these as finite groups with strict rules.

| The "Always": Invariants

An **Invariant** is a property that must be true for the system's entire life.

Chemistry

Pressure must not exceed
tank capacity.

Hospital

Dosage rate must be > 0
and $< \text{lethal limit}$.

Social Media

Follower count cannot be
negative.

| Safety: "Nothing Bad Happens"



Safety properties ensure the system never enters a forbidden state.

Example: In a banking vault, the door can only unlock IF the current time is within business hours AND the manager's key is present.

| Liveness: "Something Good Happens"

Liveness ensures the system doesn't just sit there doing nothing (Deadlock).

Example: If a passenger presses the call button, the elevator **MUST** eventually arrive.

Not just "it might", but it "eventually will".





The Workflow

How to write a spec

Step 1: Understand the Domain



You can't formalize what you don't understand.

- ✓ Talk to subject experts.
- ✓ Identify "Critical Path" features.
- ✓ Strip away the "UI fluff" to find the logic core.

| The 4-Step Specification Cycle

1

Elicit

Gather messy
requirements.

2

Model

Draw sets and states.

3

Specify

Write pre/post conditions.

4

Verify

Check for logic errors.

Step 2: Abstracting the Essence

Don't Specify:

Button color, font size, loading spinner speed, or database connection strings.

Do Specify:

Who can delete data? What happens if two users click at once? Can the balance go below zero?

| Step 3: Verification (Model Checking)

Verification is where we ask a computer to try every possible combination of inputs to find a bug.

It's like playing 1 million games of chess in 1 second to see if there's any way the king can be trapped without escape.



| The Impact of Formal Verification

10x

Reduction in critical security bugs found after launch.

0%

Chance of "Deadlock" in verified traffic control systems.

Real-Life Case: Traffic Lights



Crucial Constraints:

- ✓ **Safety:** Lights for intersecting roads cannot be Green at the same time.
- ✓ **Liveness:** Every road gets a Green light within 5 minutes.
- ✓ **Logic:** Red → Green → Yellow → Red.

| Writing the Traffic Rule

In formal math, the safety rule looks like this:

$$\neg (\text{LightA} = \text{Green} \wedge \text{LightB} = \text{Green})$$

"It is NOT the case that both Light A and Light B are Green."



The Tools of the Trade

Software to help you specify

| Popular Spec Languages



TLA+

Used by Amazon and Microsoft to find bugs in cloud systems. Great for timing and sequences.



Z Notation

Based on Set Theory.
Excellent for data-heavy systems like banks or databases.



B-Method

Allows you to generate real C code directly from the mathematical model.

Step 4: Refinement (Math to Code)



Refinement is taking a "High-Level" idea and slowly adding "Low-Level" details.

- 1 **Abstract:** "Withdraw money."
- 2 **Intermediate:** "Verify balance, subtract X, give cash."
3. **Concrete:** `if (bal >= x) { bal -= x; }`

Real-Life Case: Smart Elevator

Safety Specs:

- ✓ Motor cannot run if Door = Open.
- ✓ Elevator cannot stop between floors.
- ✓ Emergency brake engages if speed > limit.



The Elevator Invariant

We define a mathematical invariant for passengers:

Invariant: $\text{CurrentFloor} \in \{1, 2, 3, 4, 5\}$

Safety: $\text{Motor_Running} \Rightarrow \text{Door_State} = \text{Closed}$

If the motor is running, then it IMPLIES the door is closed. No exceptions.

| Where to Spend Your Time?



■ 40% Specification & Design

■ 30% Automated Verification

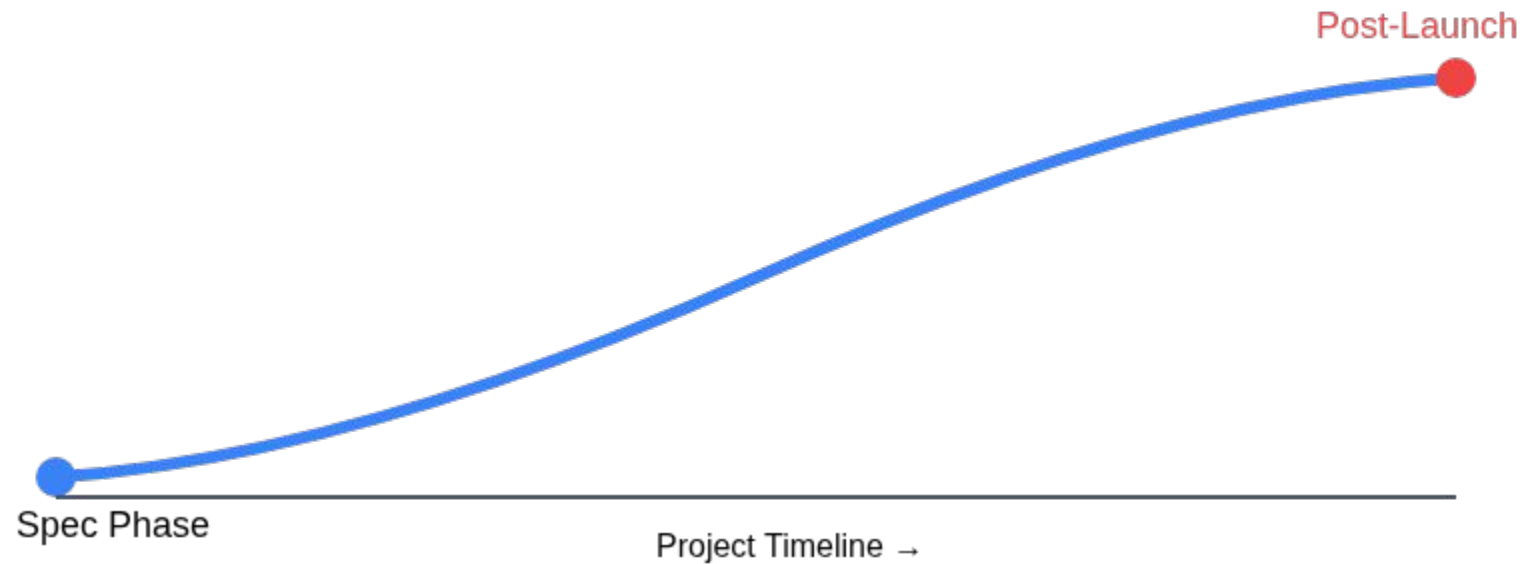
■ 30% Coding & Deployment

Traditional coding skips the first 70% and pays for it later!

| Common Pitfalls to Avoid

- ✓ **Over-specifying:** Trying to describe every tiny detail. Keep it abstract!
- ✓ **Under-specifying:** Leaving out edge cases (like "what if the power goes out?").
- ✓ **Ignoring the User:** Writing math that doesn't actually solve the real problem.
- ✓ **Math Paralysis:** Getting stuck in complexity instead of building.

| Cost of Fixing Bugs Over Time



Formal specs find bugs in the "Blue" zone, where they cost almost nothing to fix.

Summary: Why it Matters



Formal Specs aren't just for NASA. They are for anyone building systems that **must work**.

- ✓ They provide absolute clarity.
- ✓ They enable automated checking.
- ✓ They save time, money, and lives.

Questions & Answers

Thank you for your attention!

Topic: Formal Specifications for Real-Life
Problems